

Contents

Contents	2
I. Forjando Nuevos Caminos	3
II. Prólogo	4
III. Introducción	5
A. Ejercicio 0: El Juramento del Vengador	6
B. Ejercicio 1: Cuando Todo Sale Verde	7
C. Ejercicio 2: El Manual del Soldado (Métodos <i>default</i>)	8
D. Ejercicio 3: El Escudo Forjado (Clase Abstracta)	9
E. Ejercicio 4: Interfaz vs. Clase Abstracta (El Dilema Real)	10
F. Ejercicio 5: Múltiples Juramentos (Multi-Implementación)	11
G. Ejercicio 6: El Verdadero Polimorfismo (Contrato + Comportamiento Distinto)	12
Reto. "Cualquiera Puede Llevar el Escudo"	13
Ejecución	15

I. Forjando Nuevos Caminos

Para diseñar sistemas robustos como los que exige una arquitectura moderna, debes comprender las diferencias fundamentales entre las dos herramientas principales de la abstracción en programación orientada a objetos.

Regla de Oro: *Una interfaz define un comportamiento (un rol que se asume), mientras que una clase abstracta define una identidad parcial (una base de la cual se descende).*

Instrucciones Generales

- ✓ **Versión:** Java 17+ (OpenJDK).
- ✓ **Estructura:** Una clase por archivo (Puedes usar package).
- ✓ **Linter Obligatorio:** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ **Comentarios:** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ **IDE:** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ **Compilación:** Vía terminal: `javac Clase.java` y `java Clase`.

Entrega y Evaluación

- Entrega en repositorio Git.

II. Prólogo

Steve Rogers no fue elegido por su fuerza física, sino por su inquebrantable capacidad de mantenerse fiel a un juramento incluso cuando todo a su alrededor cambia, se corrompe o se desmorona. El Capitán América es, en esencia, un contrato viviente: quien lleve el escudo debe cumplir ciertos principios fundamentales — **proteger, liderar, nunca rendirse** — sin importar quién esté debajo de la máscara o qué época de la historia le toque enfrentar.

En este módulo vas a construir exactamente esa idea en código puro.

- Una **interface** es un juramento: define estrictamente *qué* debe hacer una clase, sin imponerle *cómo* debe hacerlo.
- Una **clase abstracta** es el escudo mismo: una parte ya está forjada y lista (comportamiento compartido), mientras que otra parte se deja abierta para que cada portador la complete a su propia manera.



Hasta ahora habías utilizado la herencia tradicional (Módulo 01) para declarar una relación estricta: "esto **ES** un tipo de aquello". A partir de ahora, vas a dominar una herramienta mucho más flexible y poderosa: "esto **PUEDE HACER** esto otro", sin importar de qué familia, jerarquía o rincón del sistema provenga la clase. Esto es el verdadero **polimorfismo avanzado**, la base arquitectónica que separa el código rígido del código limpio, escalable y capaz de evolucionar sin romperse ante los cambios.

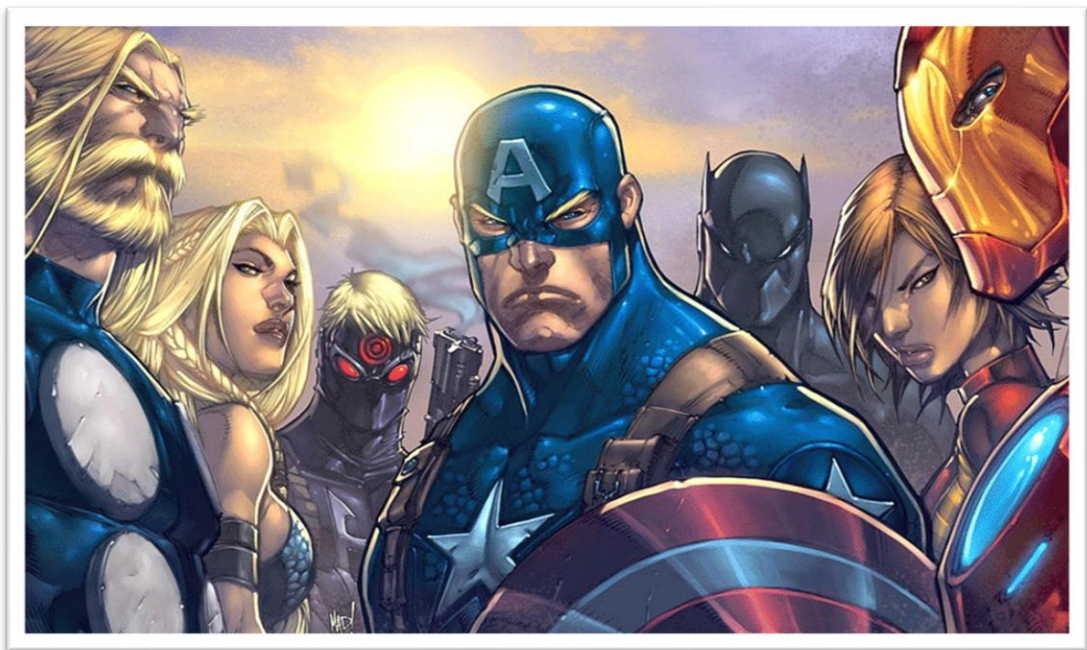
III. Introducción

¡Bienvenido al Módulo 07: El Escudo y el Juramento!

Ya dominas las bases de la programación orientada a objetos como ***extends*** y ***super()*** (vistas en el Módulo 01). Ahora darás el siguiente gran salto conceptual al conocer una segunda forma de relacionar clases basada en contratos y abstracción.

En este recorrido técnico y conceptual aprenderás:

- **Interfaces poderosas:** Qué es una interface y por qué una clase puede implementar múltiples contratos a la vez (rompiendo la limitación de la herencia simple, donde solo se permite un padre).
- **Métodos Default:** Cómo aprovechar los métodos por defecto en las interfaces para inyectar comportamiento compartido sin forzar jerarquías de clases rígidas.
- **Clases Abstractas (abstract class):** Qué son, cómo funcionan y en qué se diferencian conceptual y técnicamente de una interfaz.
- **Criterios de Diseño:** Cuándo elegir una interfaz y cuándo optar por una clase abstracta, una de las decisiones de arquitectura más importantes en cualquier desarrollo de software real.
- **Polimorfismo Real:** Cómo tratar a objetos completamente distintos como si fueran hermanos bajo un mismo paraguas, operando de forma uniforme gracias al cumplimiento de un contrato común.



A. Ejercicio 0: El Juramento del Vengador

- ✓ **Directorio:** ex0/
- ✓ **Archivo:** Avenger.java, CaptainAmerica.java
- ✓ **Funciones Autorizadas:** interface, implements, la anotación @Override
- ✓ **Método:** Any.
- ✓ **Restricción:**
 - El método definido dentro de la interfaz **no puede tener cuerpo** (no utilices la palabra clave **default**).

Objetivo

El objetivo de este ejercicio es comprender cómo funciona un contrato básico en Java (una interface).

1. **Crea una interfaz llamada Avenger:** Esta interfaz representará el contrato que todo Vengador debe cumplir. Debe declarar un método público llamado battleCry() que devuelva un valor de tipo String (es decir, el grito de batalla).
2. **Crea una clase llamada CaptainAmerica:** Esta clase debe implementar la interfaz Avenger utilizando la palabra clave implements.
3. **Implementa el contrato:** Dentro de la clase CaptainAmerica, escribe la lógica del método battleCry() utilizando la anotación @Override para asegurarte de que estás cumpliendo correctamente el contrato. El método debe retornar exactamente el texto: **"Avengers, assemble!"**.
4. **Prueba tu código:** Añade un método **main** (ya sea dentro de **CaptainAmerica** o en una clase ejecutable separada) donde crees una instancia del Capitán América y muestres por consola el resultado de invocar su grito de batalla con el formato indicado en la salida esperada.

Salida Esperada

```
Captain America says: Avengers, assemble!
```

B. Ejercicio 1: Cuando Todo Sale Verde

- ✓ **Directorio:** ex1/
- ✓ **Archivo:** Falcon.java, RosterTest.java
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** interface, implements, List<Avenger>, polimorfismo puro.
- ✓ **Restricción:**
 - *Debes recorrer la lista de Vengadores utilizando un bucle (como un **for-each**) **sin preguntar jamás de qué clase concreta es cada elemento** (está prohibido usar instanceof en este ejercicio; confía ciegamente en el contrato de la interfaz).*

Objetivo

Este ejercicio te enseñará la magia del **polimorfismo**: la capacidad de tratar a objetos que pertenecen a clases completamente distintas de exactamente la misma manera, simplemente porque ambos firman el mismo contrato.

1. **Asegura la interfaz:** Utiliza o adapta la interfaz **Avenger** creada en el ejercicio anterior (asegúrate de que tenga el método ***String battleCry()***).
2. **Reutiliza la interfaz** Avenger y CaptainAmerica de ex0, importándolas.
3. **Crea la clase de prueba (RosterTest.java):**
 - Dentro de su método main, inicializa una lista de tipo List<Avenger> (por ejemplo, utilizando ArrayList).
 - Añade a la lista una instancia de CaptainAmerica y una instancia de Falcon.
 - Recorre la lista utilizando un bucle e invoca el método battleCry() en cada elemento iterado. Observa cómo Java sabe automáticamente qué método ejecutar a pesar de que la variable de la lista es de tipo genérico Avenger.

Salida Esperada

```
Avengers, assemble!  
On your left!
```

C. Ejercicio 2: El Manual del Soldado (Métodos *default*)

- ✓ **Directorio:** ex2/
- ✓ **Archivo:** Avenger.java, WinterSoldier.java (y opcionalmente una clase con el main o integrarlo en una de ellas)
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** default (en interfaces)
- ✓ **Restricción:** El método con comportamiento por defecto (**default**) **no debe ser sobreescrito** en la clase **WinterSoldier** (es decir, debe heredar y utilizar directamente la lógica proveída por la interfaz).

Objetivo

Hasta ahora sabías que una interfaz solo contenía firmas vacías. Sin embargo, desde Java 8 es posible utilizar **métodos default**. Estos métodos permiten inyectar código e implementación real directamente en la interfaz, permitiendo que todas las clases que la implementen adquieran ese comportamiento de forma automática, sin necesidad de repetirlo ni de obligar a sobrescribirlo.

1. Actualiza la interfaz Avenger:

- Mantén el método abstracto String battleCry().
- Añade un método con la palabra clave **default** llamado **introduce()**. Este método debe ser de tipo **void** y su única tarea será imprimir por consola exactamente el siguiente texto: "I am an Avenger."

2. Crea la clase WinterSoldier:

- Haz que **WinterSoldier** implemente la interfaz **Avenger** (**implements Avenger**).
- Implementa obligatoriamente el método **battleCry()** utilizando **@Override**, haciendo que retorne el texto: **"Ready to comply... no, wait. Ready to fight."**
- **No sobrescribas** el método **introduce()** dentro de WinterSoldier; deja que actúe el mecanismo *default* de la interfaz.

3. Prueba tu código (main):

- Crea una instancia de WinterSoldier.
- Invoca primero el método introduce() (el cual heredará por defecto de la interfaz).
- Invoca después el método battleCry() e imprímelo por consola siguiendo el formato de la salida esperada.

Salida Esperada

```
I am an Avenger.  
Winter Soldier says: Ready to comply... no, wait. Ready to fight.
```


D. Ejercicio 3: El Escudo Forjado (Clase Abstracta)

- ✓ **Directorio:** ex3/
- ✓ **Archivo:** ShieldBearer.java, SamWilson.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** abstract class, métodos abstractos (abstract), métodos concretos y herencia (extends).
- ✓ **Restricción:** La clase abstracta **debe tener al menos un método ya implementado** (con cuerpo) que compartan todas las subclases, **y al menos un método abstracto** (sin cuerpo) que obligue a los hijos a completarlo a su manera.

Objetivo

En los ejercicios anteriores utilizaste **interfaces** (interface) para definir contratos puros. Ahora vas a dar el salto a una **clase abstracta** (abstract class). Una clase abstracta funciona como una plantilla a medio camino: no se puede instanciar directamente con new, pero te permite combinar **código real reutilizable** (métodos concretos) con **obligaciones personalizadas** (métodos abstractos que cada hijo debe programar obligatoriamente).

1. Crea la clase abstracta ShieldBearer (El Portador del Escudo):

- Define un método concreto (con cuerpo) llamado **raiseShield()** que imprima por consola exactamente el texto: "Shield raised.". Este comportamiento será heredado tal cual por todas las subclases.
- Define un método abstracto (sin cuerpo, usando la palabra clave abstract de tipo String llamado **motto()**). Esto obliga a cualquier clase que extienda de **ShieldBearer** a definir su propio lema.

2. Crea la clase SamWilson:

- Haz que **SamWilson** extienda de la clase abstracta mediante **extends ShieldBearer**.
- Utiliza la anotación **@Override** para implementar el método abstracto motto(), haciendo que retorne el texto: "The shield doesn't belong to a man. It belongs to an idea."

3. Prueba tu código (main):

- Crea una instancia de **SamWilson** (puedes guardarla en una variable de tipo **SamWilson** o de tipo **ShieldBearer** gracias al polimorfismo).
- Invoca primero el método heredado **raiseShield()**.
- Invoca después el método **motto()** imprimiendo su resultado por consola con el formato indicado.

Salida Esperada

```
Shield raised.  
Sam Wilson's motto: The shield doesn't belong to a man. It belongs to an idea.
```

E. Ejercicio 4: Interfaz vs. Clase Abstracta (El Dilema Real)

- ✓ **Directorio:** ex4/
- ✓ **Archivo:** DesignChoice.md

Objetivo– Demostrar tu comprensión teórica y de diseño de software respondiendo de forma clara, crítica y con tus propias palabras a tres de las preguntas más importantes en la programación orientada a objetos.

- **El problema de la múltiple identidad:** Si necesitas que una clase cumpla múltiples roles al mismo tiempo (por ejemplo, un personaje que es **Avenger**, pero también **Pilot** y además **TeamLeader**), ¿utilizarías interfaces o herencia de clases (**extends**)? Explica detalladamente **por qué** y qué limitación del lenguaje interviene.
- **El dilema de la implementación compartida:** Si necesitas compartir código base, atributos de estado y lógica real entre varias clases fuertemente emparentadas, ¿en qué momento exacto elegirías utilizar una **clase abstracta** en lugar de una interfaz?
- **La evolución de Java (Métodos Default):** Desde la versión 8, las interfaces en Java permiten añadir métodos con comportamiento por defecto (default). Analiza y argumenta: **¿Vuelve esto obsoletas a las clases abstractas?** Justifica tu respuesta explicando las diferencias técnicas clave que siguen manteniendo vivas a las clases abstractas (como el manejo de estado o constructores).

F. Ejercicio 5: Múltiples Juramentos (Multi-Implementación)

- ✓ **Directorio:** ex5/
- ✓ **Archivo:** Avenger.java, Pilot.java, SamWilson.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** implements InterfazA, InterfazB (varias a la vez)
- ✓ **Restricción:** La clase SamWilson **debe implementar obligatoriamente dos interfaces distintas al mismo tiempo**, aprovechando una de las grandes ventajas que diferencian a las interfaces de la herencia tradicional de clases (donde solo se permite un único padre).

Objetivo

Mientras que la **herencia simple (extends)** te ata a un único árbol genealógico (un objeto solo puede tener un padre), las interfaces rompen esa cadena. Una clase puede firmar tantos contratos como necesite para describir todas sus capacidades. En este ejercicio verás cómo Sam Wilson combina su identidad como Vengador con su habilidad como Piloto.

1. **Asegura la interfaz:** Utiliza o adapta la interfaz **Avenger** creada en el ejercicio anterior (asegúrate de que tenga el método ***String battleCry()***).
2. **Crea la segunda interfaz Pilot:**
 - Debe declarar un método void fly().
3. **Crea la clase SamWilson:**
 - Haz que implemente ambas interfaces utilizando una coma para separarlas: implements Avenger, Pilot.
 - Utiliza @Override para programar el método battleCry(), haciendo que retorne el texto: "On your left!".
 - Utiliza @Override para programar el método fly(), haciendo que imprima por consola exactamente el texto: "Sam Wilson is flying with the Falcon wings."
4. **Prueba tu código (main):**
 - Crea una instancia de SamWilson.
 - Invoca su método battleCry() imprimiendo el resultado por consola con el prefijo correspondiente.
 - Invoca su método fly().

Salida Esperada

```
Sam Wilson says: On your left!
Sam Wilson is flying with the Falcon wings.
```

G.Ejercicio 6: El Verdadero Polimorfismo (Contrato + Comportamiento Distinto)

- ✓ **Directorio:** ex6/
- ✓ **Archivo:** SteveRogers.java, SamWilson.java, LegacyTest.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** abstract class, uso de List<ShieldBearer>, y polimorfismo real.
- ✓ **Restricción:** Debes recorrer una única lista de objetos que pertenecen a clases distintas (**List<ShieldBearer>**) utilizando un bucle, sin preguntar explícitamente el tipo de objeto con **instanceof**. Cada objeto debe responder automáticamente a su manera al método abstracto.

Objetivo

El polimorfismo en su máxima expresión combina dos mundos:

1. **Comportamiento heredado e idéntico:** Código que ya está escrito en la clase abstracta y que todos los objetos ejecutan exactamente igual (en este caso, levantar el escudo).
2. **Comportamiento personalizado y dinámico:** Un **método abstracto** que obliga a cada clase hija a reescribirlo a su manera, logrando que una misma llamada (**motto()**) produzca resultados totalmente diferentes según el objeto que la reciba en ese momento.
3. **Asegura la clase abstracta ShieldBearer:**
 - Utiliza la clase abstracta creada en el Ejercicio 3 que contiene:
 - El método concreto void raiseShield() que imprime "Shield raised."
 - El método abstracto String motto().
4. **Crea la clase SteveRogers:**
 - Haz que extienda de ShieldBearer (extends ShieldBearer).
 - Implementa obligatoriamente el método motto() usando @Override, haciendo que retorne el texto: "I can do this all day."
5. **Reutiliza o asegura la clase SamWilson:**
 - Haz que extienda de **ShieldBearer** (extends ShieldBearer).
 - Implementa su método motto() haciendo que retorne el texto: "The shield doesn't belong to a man. It belongs to an idea."
6. **Crea la clase de prueba (LegacyTest.java):**
 - En su método main, inicializa una lista polimórfica: **List<ShieldBearer> bearers = new ArrayList<>();**
 - Añade a la lista una instancia de **SteveRogers** y una instancia de **SamWilson**.
 - Recorre la lista con un bucle (**for-each**). Para cada elemento, invoca primero **raiseShield()** e imprime inmediatamente después el resultado de su método **motto()** siguiendo el formato exacto de la salida esperada.

Salida Esperada

```
Shield raised.
I can do this all day.
Shield raised.
The shield doesn't belong to a man. It belongs to an idea.
```

Reto. "Cualquiera Puede Llevar el Escudo"

- **Directorio:** reto/, solo un archivo.
- **Dónde ejecutarlo:** en el mismo proyecto *hola-bosque* que ya tienes no crees uno nuevo.

Objetivo: en este módulo aprendiste que una interfaz define un contrato que distintas clases pueden cumplir a su manera. Ahora vas a exponer eso en el navegador: un **endpoint** que, según un parámetro, "invoca" a un portador del escudo distinto, pero todos responden al mismo contrato.

Qué vas a agregar dentro de tu clase `HolaBosqueApplication.java`, justo debajo del método `bifrostSighting()` (del Reto 06):

```
interface ShieldBearer {
    String motto();
}

class SteveRogers implements ShieldBearer {
    public String motto() {
        return "I can do this all day.";
    }
}
```

Y el nuevo método:

```
@GetMapping("/shield")
public String shieldBearer(@RequestParam String name) {
    try {
        ShieldBearer bearer;
        if (name.equalsIgnoreCase("steve")) {
            bearer = new SteveRogers();
        } else if (name.equalsIgnoreCase("sam")) {
            bearer = new SamWilson();
        } else {
            throw new IllegalArgumentException("Unknown shield bearer: " + name);
        }
        return bearer.motto();
    } catch (IllegalArgumentException e) {
        return "Error: " + e.getMessage();
    }
}

class SamWilson implements ShieldBearer {
    public String motto() {
        return "The shield doesn't belong to a man. It belongs to an idea.";
    }
}
```

No necesitas **import** nuevo — reutilizas los mismos que ya tenías (`@RequestParam`, `@GetMapping`).

Cómo probarlo

1. Guarda, reinicia el servidor.
2. Ve a esta URL en tu navegador:
`http://localhost:8080/shield?name=steve`
3. Deberías ver: ***I can do this all day.***
4. Ahora prueba con un valor negativo:
Prueba con `http://localhost:8080/shield?name=sam` y verás el mote distinto.
5. Prueba con un nombre inválido (**`?name=thanos`**) para confirmar que el try/catch sigue protegiendo el endpoint.

Qué está pasando (sin entrar en detalle todavía)

- La variable bearer está declarada como `ShieldBearer` (la interfaz), no como `SteveRogers` o `SamWilson` — eso es lo que te permite tratarlos de forma uniforme, aunque sean clases distintas.
- Esto es la misma idea que usan los frameworks profesionales (incluido Spring) todo el tiempo: programar contra interfaces, no contra implementaciones concretas, para poder cambiar el comportamiento sin tocar el código que lo usa.

Ejecución – Si no te funciona el botón de IntelliJ

```
cd "C:\Users\XXXX\Documents\hola-bosque\hola-bosque"  
.\mvnw.cmd spring-boot:run
```

Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando `javac` te dirá "**archivo no encontrado**" y te entrarán ganas de tirar el PC por la ventana.

- ✓ `cd Desktop/proyectos_java/ex00`
- ✓ `javac FtHelloForest.java` -> La Compilación (El traductor)
- ✓ `java FtHelloForest` -> La Ejecución (El lanzamiento)